Since 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Campus Network Design and Packet Trace Simulator

Computer Networks Laboratory | CSE20110

PROJECT REPORT

Department of Computer Engineering and Technology

Panel J | Batch J2 | Academic Year 2026–2027

Faculty: Dr. Uma Pujeri

GitHub Repository: https://github.com/AsHx-BiT/Campus-Network-

Ajinkya Katke | PRN 1262243726

Executive Summary

This project presents a campus network divided into four department VLANs and a companion website that visualizes packet paths through the design. In Cisco Packet Tracer, a Cisco 2911 router provides inter-VLAN routing and DHCP, a 2960 switch connects the endpoints, and a server provides HTTP and DNS at 192.168.40.10. The companion dark-theme website has a Python backend and an interactive frontend for tracing modeled ICMP, TCP, and UDP packets. The website is a teaching model: it does not send traffic to or control the Packet Tracer application.

Project at a glance

Item	Project detail
Network simulation	Cisco Packet Tracer campus LAN with four VLANs
Network devices	Router R1 (2911), switch SW1 (2960), eight PCs and one Server-PT
Network services	Router DHCP; server HTTP and DNS
Companion website	Responsive dark interface with Python standard-library backend
Local address	http://127.0.0.1:8090/
Project files	Computer_Networks_Project.pkt and website/

1. Introduction

A campus network serves users with different responsibilities. Administration, teachers, students, and library users share physical switching infrastructure, but their traffic can be organized into separate logical networks. This project uses VLANs to separate those groups and a router trunk to route traffic between VLANs when a shared service must be reached [1].

The project has two connected parts. The Packet Tracer file contains the network topology and device configuration. The Campus Packet Lab website is a companion demonstration tool: it displays the topology and models how selected packets move through switches and the router. The website is separate from Cisco Packet Tracer and does not generate live traffic in that application.

1.1 Problem statement

The project addresses how to organize a small campus LAN into department networks while retaining access to shared services such as a web page and DNS. It also provides a visual way to explain packet forwarding, VLAN boundaries, routing, and TTL changes during a classroom demonstration.

1.2 Objectives

- Create four department VLANs with separate IPv4 /24 subnets.
- Configure router-on-a-stick inter-VLAN routing and DHCP gateways.
- Provide a fixed-address campus server with HTTP and DNS services.
- Build a dark-theme website with an interactive frontend and packet-trace backend.
- Document repeatable checks for the Packet Tracer topology and website model.

2. System Requirements and Scope

The network simulation is intended to be opened in Cisco Packet Tracer 9.x. The companion web application runs locally with Python 3.10 or later and uses only the Python standard library. A modern browser is used to access its interface.

2.1 Components

Component	Role
Cisco 2911 router R1	Routes between VLAN subinterfaces and provides DHCP pools and default gateways.
Cisco 2960 switch SW1	Provides VLAN membership and forwards frames between clients, server, and router trunk.
Eight PC-PT clients	Two endpoints are placed in each department VLAN.
Server-PT at 192.168.40.10	Static server address; provides the Packet Tracer HTTP page and DNS records.
Campus Packet Lab frontend	Lets a presenter select endpoints and packet type, then inspect the modeled path.
Python backend	Serves the website and API, returns topology information, and calculates modeled traces.

2.2 Scope boundary

The Packet Tracer file is the network lab. The companion site is a separate application with its own packet model. A trace displayed by the site is not a capture from a real network and is not sent through Cisco Packet Tracer. The Packet Tracer Server-PT serves a static campus portal; it is not a general-purpose host for the Python application or a database.

3. Network Design

R1 connects to SW1 using an 802.1Q trunk. R1 has one logical subinterface and default gateway for each VLAN. SW1 connects eight PCs and the server through access ports. The server belongs to the library VLAN, so clients in other VLANs reach it through R1.

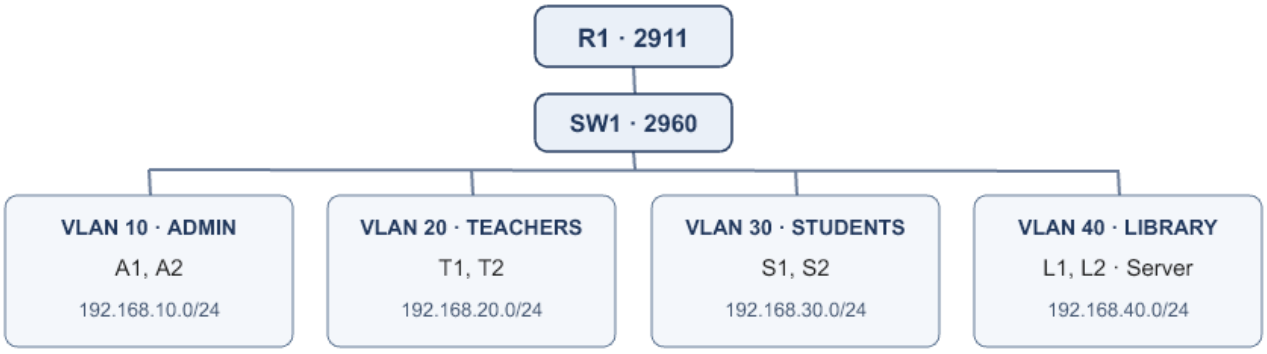


Figure 1. Campus topology and department VLANs.

3.1 IPv4 address plan

VLAN	Department	Subnet	Gateway	DHCP DNS
10	Administration	192.168.10.0/24	192.168.10.1	192.168.40.10
20	Teachers	192.168.20.0/24	192.168.20.1	192.168.40.10
30	Students	192.168.30.0/24	192.168.30.1	192.168.40.10
40	Library	192.168.40.0/24	192.168.40.1	192.168.40.10

The Server-PT uses the reserved static address 192.168.40.10. That address is excluded from the DHCP pool so it is not leased to a client. Each client pool supplies a DHCP address and the server DNS address [2].

3.2 VLAN endpoint placement

VLAN	Switch access ports	Connected endpoints
10 – ADMIN	Fa0/2–Fa0/3	A1, A2
20 – TEACHERS	Fa0/4–Fa0/5	T1, T2
30 – STUDENTS	Fa0/6–Fa0/7	S1, S2
40 – LIBRARY	Fa0/8–Fa0/10	L1, L2, Server

SW1 Fa0/1 is the router trunk. Endpoint and server ports are assigned to their department VLANs according to the saved topology.

4. Implementation

4.1 Router and switch

R1 uses router-on-a-stick routing. Each GigabitEthernet0/0 subinterface tags traffic with its VLAN ID and uses the .1 address in that VLAN as its gateway. The parent interface GigabitEthernet0/0 must be enabled for the subinterfaces and trunk to operate. DHCP pools provide client addresses and DNS information. SW1 defines VLANs 10, 20, 30, and 40, assigns endpoint ports, and trunks the router link [1].

```
interface GigabitEthernet0/0.10
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
! Repeat for VLANs 20, 30, and 40
```

The completed project enables R1 GigabitEthernet0/0, which was shut in the supplied configuration. Explicit VLAN definitions and a DHCP exclusion for the server address are also present in the completed file.

4.2 Server services and campus portal

The Packet Tracer Server-PT has the fixed address 192.168.40.10. HTTP and DNS are enabled. The DNS names www.school.com and www.school.local point to the server; the misspelled www.shool.local record was corrected. The HTTP service serves the dark Campus Connect portal from the project frontend. DNS provides name-to-address mapping for the portal [4].

4.3 Companion website frontend and backend

The frontend is a responsive dark-theme interface. A user selects a source, destination, protocol, and, where needed, destination port. The site offers example flows for a web request, an inter-VLAN ICMP packet, and a DNS request. Its inspector shows packet addresses, VLAN and interface at each modeled hop, destination port, and TTL changes.

The backend is a Python HTTP server using the standard library. It serves the static frontend and exposes API endpoints for health, topology, packet tracing, and recent in-memory traces. The site keeps up to 25 recent traces in memory. It does not use a database, control Packet Tracer, or send packets onto the campus network.

Endpoint	Purpose
GET /health	Reports whether the local web backend is running.
GET /api/topology	Returns devices, links, VLANs, and modeled services.
POST /api/trace	Calculates a modeled path for a source, destination, protocol, and port.
GET /api/traces	Returns the most recent in-memory trace records.

5. Packet Tracing Workflow

The website calculates a route from the saved campus model. When two endpoints share a VLAN, the model forwards between them through SW1. When the destination is in another VLAN, the modeled route goes through SW1, R1, and SW1 to reach the destination. A routed hop decrements the modeled IPv4 TTL by one.

5.1 Example cross-VLAN web request

1. Choose A1 in VLAN 10 as the source and Server in VLAN 40 as the destination.
2. Select TCP and destination port 80 to represent an HTTP request.
3. Run Trace and inspect the route, VLAN/interface details, and TTL change.

```
A1 (VLAN 10) → SW1 → R1 → SW1 → Server (VLAN 40)
Modeled result: TCP/80, four network hops, TTL 64 → 63
```

This example is a result from the companion site model. Cisco Packet Tracer also offers a Simulation Mode for observing packet movement in its own network simulation [3], but that runtime was not used to generate these website results.

5.2 Other modeled flows

Flow	Expected model behavior
A1 → A2, same VLAN	Forward through SW1 without R1; modeled TTL remains 64.
T1 → A1, different VLAN	Route through R1 between VLAN 20 and VLAN 10; modeled TTL decreases 64 to 63.
S1 → Server, UDP/53	Model a DNS request to 192.168.40.10.

6. Testing and Results

The companion website and API were checked locally. These results describe the website model and its HTTP responses; they are not live packet tests inside Cisco Packet Tracer.

Check	Observed result	Evidence scope
Backend health	GET /health returned a healthy response.	Local website
Topology API	Returned 11 modeled devices, 10 links, and four VLANs.	Application model
Cross-VLAN TCP	A1 to Server on TCP/80: SW1 → R1 → SW1; four modeled network hops; TTL 64 to 63.	Application model
Same-VLAN path	A1 to A2 used SW1 without an R1 hop; TTL stayed 64.	Application model
DNS request model	S1 to Server over UDP/53 used 192.168.40.10.	Application model
Invalid port	Port 70000 was rejected with HTTP 400.	API validation
Packet Tracer runtime	No live simulation or CLI test was run in Packet Tracer here.	Not verified

The results show that the web model reflects the intended VLAN routing logic and rejects invalid input. The Packet Tracer file contains the configured network, but live DHCP leases, pings, DNS resolution, and browser access should be demonstrated in Packet Tracer before reporting those runtime checks as completed.

7. Running and Demonstrating the Project

7.1 Start the companion website on Windows

```
cd website
python server.py
```

Open http://127.0.0.1:8090/ in a browser on the same computer and leave the server terminal running. Python 3.10 or later is required. A Docker option is documented in the project README.

7.2 Open and verify the Packet Tracer file

1. Open Computer_Networks_Project.pkt in Cisco Packet Tracer 9.x and allow links to initialize.

2. On each PC, open Desktop → IP Configuration and select DHCP. Check its subnet, .1 gateway, and DNS address 192.168.40.10.

3. From a PC command prompt, run ipconfig and ping its gateway and 192.168.40.10.

4. Open the PC web browser and visit http://www.school.com or http://192.168.40.10.

5. On R1, check show ip interface brief, show ip dhcp binding, and show ip route. On SW1, check show vlan brief and show interfaces trunk.

6. Run the A1-to-Server TCP/80 trace in the companion site and explain that the path is modeled by the website.

8. Limitations and Future Work

8.1 Limitations

- Website traces are calculated from a model; they are not captures and do not interact with Cisco Packet Tracer.
- The website stores only the latest 25 traces in memory; history resets when the server restarts.
- Packet Tracer Server-PT hosts a static HTTP portal and DNS records; it is not a general application backend or database.
- Live Packet Tracer execution and end-to-end device tests remain to be verified in the Packet Tracer application.

8.2 Future improvements

- Add subnet, ACL, and firewall scenarios to the trace model.
- Add exportable trace reports and persistent trace history.
- Capture live Packet Tracer DHCP, ping, DNS, HTTP, and trunk checks for the final demonstration.

9. Conclusion

The project combines a four-VLAN campus network in Cisco Packet Tracer with a local web tool that explains modeled packet paths. R1 provides inter-VLAN routing and DHCP, SW1 connects department endpoints, and the server provides HTTP and DNS. The companion frontend and Python backend make the design easier to demonstrate, while the report distinguishes modeled traces from live network traffic. Before presenting, run the listed DHCP, ping, DNS, HTTP, and show-command checks in Packet Tracer and report only results observed in that application.

References

[1] Cisco, “Configure Inter VLAN Routing with the Use of an External Router,” Cisco Support. https://www.cisco.com/c/en/us/support/docs/lan-switching/inter-vlan-routing/14976-50.html

[2] R. Droms, “Dynamic Host Configuration Protocol,” RFC 2131, IETF, Mar. 1997. https://www.rfc-editor.org/rfc/rfc2131

[3] Cisco Networking Academy, “Cisco Packet Tracer.” https://www.netacad.com/catalogs/learn/cisco-packet-tracer

[4] P. Mockapetris, “Domain Names—Concepts and Facilities,” RFC 1034, IETF, Nov. 1987. https://www.rfc-editor.org/rfc/rfc1034

[5] Project files: Computer_Networks_Project.pkt, packet-tracer/README.md, and website/README.md.